Form and input components

24C. Form & Input Components

Problems: 9 (Q 24.14 – Q 24.22)

Q 24.14 Autocomplete / Typeahead Search

The most frequently asked machine coding problem.

JSX Copy

```jsx
function Autocomplete({ fetchSuggestions, onSelect }) {
  const [query, setQuery] = useState('');
  const [suggestions, setSuggestions] = useState([]);
  const [highlightedIndex, setHighlightedIndex] = useState(-1);
  const [isOpen, setIsOpen] = useState(false);
  const [loading, setLoading] = useState(false);
  const controllerRef = useRef(null);

  useEffect(() => {
    if (!query.trim()) { setSuggestions([]); return; }
    controllerRef.current?.abort();
    controllerRef.current = new AbortController();

    const timer = setTimeout(async () => {
      setLoading(true);
      try {
        const results = await fetchSuggestions(query, controllerRef.curren
        setSuggestions(results);
        setIsOpen(true);
```

```
    } finally { setLoading(false); }
  }, 300);

  return () => clearTimeout(timer);
}, [query]);

function handleKeyDown(e) {
  if (!isOpen) return;
  if (e.key === 'ArrowDown') {
    e.preventDefault();
    setHighlightedIndex(prev => Math.min(prev + 1, suggestions.length -
  }
  if (e.key === 'ArrowUp') {
    e.preventDefault();
    setHighlightedIndex(prev => Math.max(prev - 1, 0));
  }
  if (e.key === 'Enter' && highlightedIndex >= 0) {
    onSelect(suggestions[highlightedIndex]);
    setIsOpen(false);
    setQuery(suggestions[highlightedIndex].name);
  }
  if (e.key === 'Escape') setIsOpen(false);
}

return (
  <div className="autocomplete" role="combobox" aria-expanded={isOpen}>
    <input value={query} onChange={(e) => setQuery(e.target.value)}
      onKeyDown={handleKeyDown} onFocus={() => suggestions.length && set
      aria-autocomplete="list"
      aria-activedescendant={highlightedIndex >= 0 ? `suggestion-${highl
    {loading && <span className="loading-indicator">Loading ...</span>}
    {isOpen && suggestions.length > 0 && (
      <ul role="listbox">
        {suggestions.map((item, i) => (
          <li key={item.id} id={`suggestion-${i}`} role="option"
            aria-selected={i === highlightedIndex}
            className={i === highlightedIndex ? 'highlighted' : ''}
            onClick={() => { onSelect(item); setIsOpen(false); setQuery(
            <HighlightMatch text={item.name} query={query} />
          </li>
        ))}
```

```
  );
}

function HighlightMatch({ text, query }) {
  const parts = text.split(new RegExp(`(${query})`, 'gi'));
  return parts.map((part, i) =>
    part.toLowerCase() === query.toLowerCase() ? <mark key={i}>{part}</mar
  );
}
```

Edge cases: Empty results ("No results found"), API errors (retry), very long text (truncate), click outside to close.

Q 24.15 Multi-select Dropdown with Search

Multiple items selectable, selected shown as chips, search filters options, remove with × or Backspace. `aria-multiselectable="true"` on listbox.

Q 24.16 OTP Input Component

JSX Copy

```
function OTPInput({ length = 6, onComplete }) {
  const [values, setValues] = useState(Array(length).fill(''));
  const inputsRef = useRef([]);

  function handleChange(index, value) {
    if (!/^\d*$/.test(value)) return;
    const next = [...values];
    next[index] = value.slice(-1);
    setValues(next);
    if (value && index < length - 1) inputsRef.current[index + 1].focus();
    if (next.every(v => v)) onComplete(next.join(''));
```

```
  function handleKeyDown(index, e) {
    if (e.key === 'Backspace' && !values[index] && index > 0)
      inputsRef.current[index - 1].focus();
  }

  function handlePaste(e) {
    const pasted = e.clipboardData.getData('text').slice(0, length);
    if (!/^\d+$/.test(pasted)) return;
    const next = pasted.split('').concat(Array(length).fill('')).slice(0,
    setValues(next);
    inputsRef.current[Math.min(pasted.length, length - 1)].focus();
    if (next.every(v => v)) onComplete(next.join(''));
  }

  return (
    <div className="otp-input" onPaste={handlePaste}>
      {values.map((val, i) => (
        <input key={i} ref={el => inputsRef.current[i] = el} value={val}
          onChange={e => handleChange(i, e.target.value)} onKeyDown={e =>
          maxLength={1} inputMode="numeric" autoComplete={i === 0 ? 'one-t
      ))}
    </div>
  );
}
```

Q 24.17 Multi-step Form with Validation

JSX Copy

```
function MultiStepForm() {
  const [step, setStep] = useState(0);
  const [formData, setFormData] = useState({ name: '', email: '', address:

  const steps = [
    { component: PersonalInfo, validate: (d) => d.name && d.email },
    { component: AddressInfo, validate: (d) => d.address },
    { component: PaymentInfo, validate: (d) => d.payment },
```

```
  function next() { if (steps[step].validate(formData)) setStep(s => s + 1

  const StepComponent = steps[step].component;

  return (
    <div>
      <ProgressBar current={step} total={steps.length} />
      <StepComponent data={formData} onChange={setFormData} />
      <div className="actions">
        {step > 0 && <button onClick={() => setStep(s => s - 1)}>Back</but
        {step < steps.length - 1
          ? <button onClick={next}>Next</button>
          : <button onClick={() => submitForm(formData)}>Submit</button>}
      </div>
    </div>
  );
}
```

Q 24.18 – Q 24.22 (Date Picker, Color Picker, Range Slider, Rich Text Editor, File Uploader)

Date Picker (24.18): Calendar grid with month/year navigation. Keyboard: arrow keys between days, Enter to select. Hardest part: generating the calendar grid correctly (first day padding, days in month).

Color Picker (24.19): Hue slider + saturation/brightness 2D picker + hex/RGB inputs. Use HSL internally, convert to hex for output.

Range Slider (24.20): Single or dual knob. Track mouse/touch events, convert pixel → value. `role="slider"`, `aria-valuenow`, `aria-valuemin`, `aria-valuemax`.

Rich Text Editor (24.21): `contenteditable` div with `document.execCommand` for basic formatting. For production, use TipTap or Slate.

File Uploader (24.22):

```jsx
function FileUploader({ maxSize = 5 * 1024 * 1024, accept = 'image/*' }) {
  const [files, setFiles] = useState([]);
  const [dragOver, setDragOver] = useState(false);

  function handleFiles(fileList) {
    const valid = Array.from(fileList).filter(f => {
      if (f.size > maxSize) { alert(`${f.name} exceeds limit`); return fal
      return true;
    });
    setFiles(prev => [...prev, ...valid.map(f => ({
      file: f,
      preview: f.type.startsWith('image/') ? URL.createObjectURL(f) : null
      progress: 0, status: 'pending',
    }))]);
  }

  return (
    <div className={`dropzone ${dragOver ? 'active' : ''}`}
      onDragOver={(e) => { e.preventDefault(); setDragOver(true); }}
      onDragLeave={() => setDragOver(false)}
      onDrop={(e) => { e.preventDefault(); setDragOver(false); handleFiles
      <p>Drag files here or <label>browse<input type="file" hidden multipl
        onChange={(e) => handleFiles(e.target.files)} /></label></p>
      {files.map((f, i) => (
        <div key={i} className="file-preview">
          {f.preview && <img src={f.preview} alt="" width={60} height={60}
          <span>{f.file.name}</span>
          <progress value={f.progress} max={100} />
        </div>
      ))}
    </div>
  );
}
```

QUICK CHECK — no marks, just for you.

1. In the OTPInput component, the user has typed digits into all six boxes, then clicks into box 4, deletes its digit, and presses Backspace once more while box 4 is now empty. According to the handleKeyDown implementation shown, what happens?

- Focus moves back to box 3, leaving box 3's digit intact
- Box 3's digit is deleted and focus stays on box 4
- Box 3's digit is deleted and focus moves back to box 3

Check answer

2. In MultiStepForm, a user is on the Address step, leaves the address field empty, and clicks Next. Based only on the code shown, what is the result?

- The step does not advance, and no validation message is shown to the user
- The step advances, and the address is validated later at submit time
- The step does not advance, and an error message appears under the address field
- The step advances, but the Submit button on the final step stays disabled

Check answer

3. In the Autocomplete component, the catch block only calls setSuggestions([]) when the caught error's name is not 'AbortError'. Why is that guard there?

- An AbortError can only fire after the debounce timer is cleared, so setSuggestions would run on an unmounted component
- An AbortError means a newer keystroke deliberately cancelled that request, so wiping the list would discard results the newer request is about to deliver
- Without the guard, the AbortController would rethrow the error and crash the component tree

An AbortError signals the network is offline, so the component should keep the list and retry the same request later

Check answer

Previous
Approach and core components

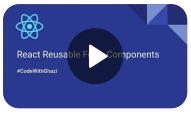
Next
Layout and data display

LEARN MORE

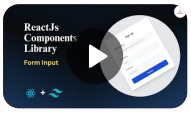
ARTICLES & DOCS

- <input> (MDN / React)
  react.dev
- React Hook Form
  react-hook-form.com

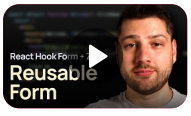
VIDEOS



- Reusable input field component
  YouTube



- Reusable form input UI component
  YouTube



- Reusable create/edit form (RHF + Zod)
  YouTube

Hand-picked references for this lesson.